

1. Test accuracy: 94.37%
2. Best validation accuracy: 39.3%, reached in epoch 9
Dense test accuracy: 44.9%
3.
 - a. For the naive method, I noticed that the validation error was 2.46 which is lower than the training error 2.70. Usually the training error is lower than the validation error, so this seems a bit unusual. If we tried the naive method on different datasets, I wouldn't expect this pattern to happen every time. It looks more like an anomaly specific to this particular data split. The naive method is simple and doesn't try to memorize the data very well. For example, it might just predict that tomorrow's value will be the same as today's. Since the method is so basic, it's not likely to fit the training data perfectly. The lower validation error could just be due to some random chance in how the data is split or maybe the validation set has patterns that are easier to predict. The training set might include some more difficult or noisy data, which could make it harder for the model to perform well on it. In most cases with the naive method, I would expect the training and validation errors to be pretty close, with the training error usually being a bit lower or about the same as the validation error.
 - b. For the fully connected network, I saw that the training error kept getting lower (from around 2.75 to 2.05), while the validation error decreased at first but then started to go up again, with the lowest point being around 2.56. If we used this fully connected network on other similar datasets, I would expect the training error to always be lower than the validation error. Fully connected networks are much better at memorizing the training data because they have a lot of capacity. The pattern we're seeing suggests that the model is starting to overfit after about 4 epochs, meaning it's learning specific details of the training data that don't apply to the validation data. This kind of behavior where training error keeps dropping while validation error starts to increase is a classic sign of overfitting. Neural networks, especially complex ones with many parameters, are prone to this unless we take steps to prevent it. I'd expect to see this same pattern across similar datasets, but the size of the gap between training and validation errors could change depending on things like how much data we have, how noisy the data is, and the complexity of the network.
 - c. For the Recurrent Neural Network (RNN), I noticed a pattern similar to the fully connected network, but with an even larger gap between training and validation performance. The training error decreased consistently to around 2.15, while the validation error plateaued at about 2.55. If we used the RNN on other similar

datasets, I'd expect the training error to always be lower than the validation error, often with a bigger gap than we saw with the fully connected network. RNNs have special connections that allow them to capture complex patterns over time, making them much better at handling sequence data. Because of their memory capacity, RNNs can learn patterns from earlier time steps, which is a big advantage for time series tasks. However, this also makes them more prone to overfitting. In the graph, we can see the training error keeps improving without the validation error improving in the same way. This suggests the model is memorizing specific patterns in the training data that don't apply to the validation set. RNNs are particularly prone to overfitting on time series data because they can memorize these detailed temporal patterns instead of learning more generalizable trends. I'd expect this same pattern to hold across similar datasets, with the gap between training and validation error likely being larger than what we saw with the fully connected network. The size of the gap will depend on factors like dataset size, data noise, the network's complexity, and, for RNNs, the complexity of the temporal patterns in the data.

4. Using the tanh activation function for the output layer in a temperature prediction network would cause several issues. First, the tanh function limits outputs to a range between -1 and 1, which is far too narrow for temperature values. This would prevent the network from predicting accurate temperature values. If the predicted temperature is supposed to be greater than 1, the tanh function would force the network to output a value no larger than 1. This leads to systematic errors in predictions. Additionally, tanh compresses values near its limits, making it hard for the network to accurately predict temperatures near -1 or 1 which reduces precision. The training data would likely involve real-world temperatures far outside the tanh range, which could cause problems during training as the network's gradients might quickly saturate. The loss function, like MSE, would also struggle to reduce the error, leading to large, persistent mistakes the network can't correct. The identity function is a better choice for this task because it doesn't limit the output range and allows the network to predict any real number, which is essential for temperature prediction. This ensures that the network can handle the wide range of temperature values that might come up in different situations and seasons.